

CMGen User Guide

Version V-2023.12, December 2023



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Audience	8
Related Publications	8
Release Notes	9
Conventions	9
Customer Support	10

1. CMGen Flow	11
Characterization Flow	11
Tool Compatibility	12

2. CMGen Input Files	13
SPICE Netlist File	13
SPICE Simulation Library	14
Liberty File	14
CMGen Setup File	16

3. Invoking CMGen and the CMGen Setup Commands	17
Invoking CMGen	17
Example CMGen Setup File	18
Required Setup Commands	18
set liberty_file	18
set cell_netlist	19
set spice_model	19
set log_file	19
set partition_multibit_netlist	19
set output_dir	20
set output_dir	20
Optional Commands	20
set cell_ground	23
set cell_list_file	23

Contents

set cell_power	23
set constrain_scan_enable	24
set constrain_set_reset	24
set defect	24
set delay_variation	24
set device_name_prefix	25
set device_name	25
set extra_hspice_options	25
set extra_finesim_options	25
set hspice_runelevel	26
set hspice_wrapper_command	26
set ignore_device_name	26
set input_dir	26
set keep_testbench	27
set numcores	27
set open	27
set open_range	28
set pin_constraint	28
set pin_ignore	28
set keep_waveform	28
set short	29
set short_range	29
set skip_single_input_seq_cells	29
set spice_resim_count	30
set spice_simulator	30
set threshold_voltage	30
set temporary_storage	31
set timing_margin	31
set timing_resolution	31
set tolerant_missing_tr_file	32
set transistor_open_defect_device	32
set transistor_open_defect_location	32
set transistor_open_defect_resistor_value	32
set transistor_short_defect_device	33
set transistor_short_defect_location	33
set transistor_short_defect_resistor_value	33
set udd_dir_name	33
set unspecified_device_handling	34
set vector	34

Contents

set verilog_netlist	34
set voltage_variation	35
set zip_waveform	35
set include_transistor_instance	35
set exclude_transistor_instance	35
set create_ctm_for_0injection	36
set create_ctm_for_0det	36
set create_defect_csv file	36
set create_exhaustive_ctm	37
set outdir	37
set merge_ctm_tables	37
set skip_buffer_inverter_cells	37
set keep_single_ctm_file	38
set netlist_format DSPF	38
set reuse_subcell_ctm	38
set report_defect_detection_info	39
set separate_static_dynamic_simulation	39
set max_defect_by_user	39
set max_defects_per_simulation	40
set open_defect_class	40
set outdir_based_on_cellname	40
set spice_netlist_checker	41
Compute Farm Commands	41
set farm	41
set farm_type	41
Optional Farm-Specific Commands	42
set farm_option	42
set farm_project	42
set farm_resource	43
set active_job_limit	43
Setting Up Farm-Based Run Scripts	43
4. Defect Site Selection	44
R/C Defect Selection	44
Transistor Defects	46
User-Defined Defects	48
Enabling User-Defined Defects	48

Logical UDD File Syntax	49
Physical UDD File Syntax	49
5. Defect Detection	52
Liberty Files for Simulation Input	52
SPICE Output Files	53
Static Measure Files	53
Dynamic Transient Files	53
Testbench Files	54
SPICE .lis File	54
Waveform Analysis	54
Static Defect Analysis	55
Dynamic Defect Analysis	56
Multiple Output Support	57
Sequential Cell Support	58
6. Simulation Performance Turning	59
Machine Resource Provisioning	59
Defect Targeting	60
SPICE Simulator Accuracy	60
Pin Count	60
7. CM File Description	62
Header	62
Defect Description	63
Defect Detection Tables	64
8. CM Validation Flow	66
Introduction	66
CM Validation Specific Commands	67
CNOD Specific Commands	68
set cnod_defect_location	68
set enable_cnod_detection_table	69

Contents

set cell_instance_file69

Example of a cmval.setup Run Script69

Generating the Output70

About This Guide

The CMGen User Guide describes the usage and methodology for the CMGen utility. It is intended to be used with the existing TetraMAX cell-aware ATPG technology. This document assumes you understand how to use TetraMAX ATPG to generate test patterns as described in TetraMAX Online Help.

This manual provides background material on design-for-test (DFT) concepts, especially test terminology and scan design techniques. You can obtain more information on TetraMAX ATPG and TetraMAX II ATPG features and commands by accessing TetraMAX Help.

These topics are available:

- [Audience](#)
- [Related Publications](#)
- [Release Notes](#)
- [Conventions](#)
- [Customer Support](#)

Audience

This document is intended for design engineers who have ASIC design experience and some exposure to testability concepts and strategies.

This document is also useful for test engineers who incorporate the test vectors produced by TetraMAX® ATPG into test programs for a particular tester or who work with DFT netlists.

Related Publications

For additional information about TetraMAX ATPG and TetraMAX II ATPG, see Documentation on the Web, which is available through SolvNet® at the following address:

<https://solvnet.synopsys.com/DocsOnWeb>

You might also want to read the documentation for the following related Synopsys products: DFTMAX and Design Compiler®.

Release Notes

Information about new features, enhancements, changes, known limitations, and resolved Synopsys Technical Action Requests (STARs) is available in the TetraMAX ATPG and TetraMAX II ATPG Release Notes on the SolvNet site.

To see the TetraMAX ATPG and TetraMAX II ATPG Release Notes:

1. Go to the SolvNet Download Center located at the following address:

<https://solvnet.synopsys.com/DownloadCenter>

2. Select TetraMAX ATPG, and then select a release in the list that appears.

Conventions

The following conventions are used in Synopsys documentation.

Convention	Description
<code>Courier</code>	Indicates command syntax.
<i>Italic</i>	Indicates a user-defined value, such as <i>object_name</i> .
Bold	• Within syntax and examples, indicates user input—text you type verbatim. • Indicates a graphical user interface (GUI) element that has an action associated with it.
[]	Denotes optional parameters, such as: <code>write_file [-f filename]</code>
...	Indicates that parameters can be repeated as many times as necessary: <code>pin1 pin2 ... pinN</code>
	Indicates a choice among alternatives, such as <code>low medium high</code>
\	Indicates a continuation of a command line.
/	Indicates levels of directory structure.
Edit > Copy	Indicates a path to a menu command, such as opening the Edit menu and choosing Copy .
Ctrl+C	Indicates a keyboard combination, such as holding down the Ctrl key and pressing the C key.

Customer Support

Customer support is available through the Synopsys SolvNet customer support website and by contacting the Synopsys support center.

Accessing SolvNet

The SolvNet support site includes an electronic knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. The site also gives you access to a wide range of Synopsys online services, which include downloading software, viewing documentation, and entering a call to the Support Center.

To access the SolvNet site:

1. Go to the web page at <https://solvnet.synopsys.com>.
2. If prompted, enter your user name and password. (If you do not have a Synopsys user name and password, follow the instructions to register.)

If you need help using the site, click **Help** on the menu bar.

Contacting Synopsys Support

If you have problems, questions, or suggestions, you can contact Synopsys support in the following ways:

- Go to the Synopsys [Global Support](#) site on synopsys.com. There you can find e-mail addresses and telephone numbers for Synopsys support centers throughout the world.
- Go to either the Synopsys SolvNet site or the Synopsys Global Support site and [open a case online](#) (Synopsys user name and password required).

1

CMGen Flow

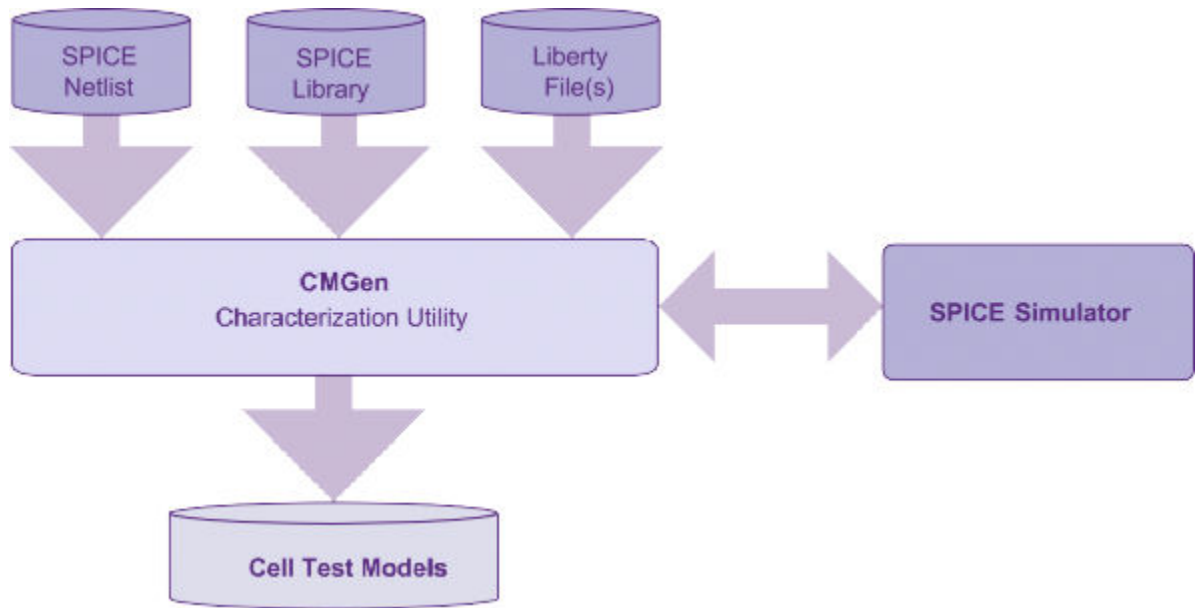
The process of creating accurate behavioral defect models, also known as cell test models (CTMs), is the foundational first step of the cell-aware test methodology. CMGen provides the simulation parameters and defect injection locations required for simulation using Synopsys SPICE simulators. The simulation results are post-processed by CMGen based on key waveform comparison points that annotate both the activation condition and the final output behavior of cell internal defects to the CM.

Characterization Flow

As shown in Figure 1, the characterization flow requires the following input files:

- SPICE netlist
- SPICE library file
- Liberty file (*.lib file used for timing calculations)
- CMGen setup file

Figure 1 Characterization for Simulation



These input files are described in detail in [CMGen Input Files](#).

Tool Compatibility

Note that the following tool versions are required to run CMGen:

- Python v2.6 or v2.7
- HSPICE 2017.03-SP2 or later versions
- FineSim 2017.03-SP2-1 or later versions

2

CMGen Input Files

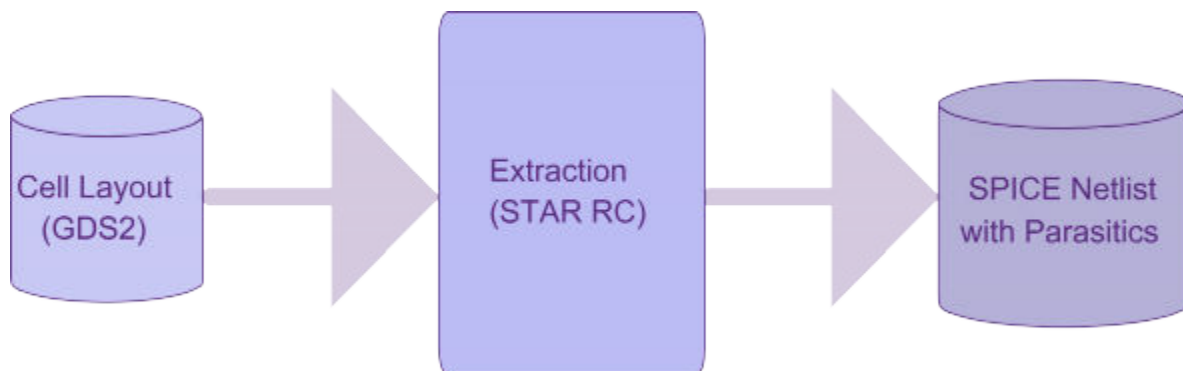
The CMGen characterization flow requires a set of input files described in the following topics:

- [SPICE Netlist File](#)
- [SPICE Simulation Library](#)
- [Liberty File](#)
- [CMGen Setup File](#)

SPICE Netlist File

The SPICE netlist is comprised of extracted parasitics generated from a layout-based extraction tool such as Synopsys StarRC. Although many strategies and options are available for creating netlists, this document describes the process of creating a netlist specifically for CMGen. Ideally, the CMGen netlist can also be used for accurate timing simulation. In most cases, the extraction is based on the process and device information specified by the library creator.

Figure 2 SPICE Netlist Creation Process



Accurate physical diagnostics requires additional physical bounding box information. This information is available from standard extraction tools, but it might not be included in existing flows.

For physical diagnostics, the following StarRC commands are required:

- `NETLIST_TAIL_COMMENTS: YES` – Outputs geometric information for parasitic resistors as comments in the netlist
- `EXTRA_GEOMETRY_INFO: NODE RES` – Reports the bounding boxes for internal nodes in the SPICE netlist

SPICE Simulation Library

The SPICE simulation library contains modeling and process information used in a SPICE simulation. A vendor-supplied SPICE library file can contain multiple process corners. You must select a single corner for use in the CMGen simulation. This corner must also match the liberty file. The library and process are defined in an 'include' file format for use with CMGen. An example spice.inc file accepted by CMGen is as follows:

```
.lib 'hspice_a_0.85V_1.5V_model.hlib' tt_allDevices  
.lib <library_location> <process_corner>
```

Note:

If you are an experienced SPICE user, you can include additional SPICE simulator options in CMGen simulation using the contents of the include file. For general users, there are no other required options for library characterization.

Liberty File

A liberty file contains the timing and environmental variables used by CMGen during a SPICE simulation. This file is created through SPICE timing characterization of the standard cell library. This means the CMGen simulation parameters should match the values expected from the standard cell designer. All values required for HSPICE simulation are derived from the liberty file.

The syntax for a liberty file is as follows:

```
library ("saed90nm_typ") {  
  technology ( "cmos" ) ;  
  time_unit : "lns";  
  voltage_unit : "1V";  
  capacitive_load_unit (1 . 000000, "ff") ;  
  slew_derate_from_library : i0000000;  
  nom_voltage : i2000000;  
  nom_temperature : 25.0000000;  
  nom_process : i0000000;  
  
  cell (BUFF16) {  
    pin (ZN) {  
      related_power_pin : "VDD";  
    }  
  }  
}
```

```

    related_ground_pin : "VSS";
    direction : "output";
    timing () {
        related_pin : "INP";
        timing_sense : "negative_unate";
        cell_rise ("del_1_7_7") {
            index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, i024");
            index_2 ("1, 120, 240, 416, 832, 1664, 3328");
            values("0.0166667, 0.0407280, 0.0622035, 0.0935596, 0.1681384,
                0.3198609, 0.6191900", \
                "0.0193158, 0.0488449, 0.0707030, 0.1019445, 0.1756194,    0.3240124,
                0.6264584", \
                "0.0230880, 0.0591263, 0.0862821, 0.1193877, 0.1925797,    0.3397357,
                0.6386753", \
                "0.0282652, 0.0723099, 0.1048429, 0.1471403, 0.2274504,    0.3730648,
                0.6674810", \
                "0.0332926, 0.0891553, 0.1298676, 0.1813218, 0.2814508,    0.4438384,
                0.7363688", \
                "0.0373731, 0.1115787, 0.1642250, 0.2272435, 0.3511899,    0.5523794,
                0.8794857", \
                "0.0380690, 0.1347593, 0.2034780, 0.2842900, 0.4385558,    0.6841428,
                i0937415");
        }
        rise_transition ("del_1_7_7") {
            index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, i024");
            index_2 ("1, 120, 240, 416, 832, 1664, 3328");
            values("0.0243203, 0.0657096, 0.1118269, 0.1812766, 0.3494364,
                0.6785375, i3543956", \
                "0.0315248, 0.0749576, 0.1180950, 0.1854869, 0.3471728,    0.6819340,
                i3399754", \
                "0.0443022, 0.0950792, 0.1373385, 0.2001432, 0.3585105,    0.6820720,
                i3422665", \
                "0.0686330, 0.1266714, 0.1754206, 0.2392286, 0.3877765,    0.7022994,
                i3506110", \
                "0.1129237, 0.1814553, 0.2359783, 0.3096219, 0.4647602,    0.7634380,
                i3821954", \
                "0.2077429, 0.2779008, 0.3421530, 0.4247044, 0.6029360,    0.9130643,
                i5143839", \
                "0.4141416, 0.4657113, 0.5369154, 0.6354449, 0.8365258, i1872625,
                i8149783");
        }

        cell_fall ("del_1_7_7") {
            index_1("0.016, 0.032, 0.064, 0.128, 0.256, 0.512, i024");
            index_2 ("1, 120, 240, 416, 832, 1664, 3328");
            values("0.0177761, 0.0415416, 0.0629784, 0.0941033, 0.1668986, 0.3128899,
                0.6136891", \
                "0.0208438, 0.0496906, 0.0713057, 0.1023731, 0.1759762,    0.3208643,
                0.6124992", \
                "0.0249761, 0.0606785, 0.0871162, 0.1197825, 0.1924125,    0.3390637,
                0.6285529", \
                "0.0304181, 0.0748927, 0.1077863, 0.1489438, 0.2280930,    0.3724585,
                0.6641067", \

```

```
"0.0380732, 0.0930281, 0.1353906, 0.1852370, 0.2846674, 0.4444371,  
0.7323311", \  
"0.0469892, 0.1188884, 0.1705124, 0.2336786, 0.3561132, 0.5562348,  
0.8760070", \  
"0.0583968, 0.1513178, 0.2159075, 0.2963411, 0.4518778, 0.6995950,  
i0992192");  
  
}fall_transition ("del_1_7_7") {index_1("0.016, 0.032, 0.064,  
0.128, 0.256, 0.512, i024");index_2 ("1, 120, 240, 416, 832, 1664,  
3328");values("0.0180155, 0.0539996, 0.0936655, 0.1528141, 0.2932774,  
0.5775458, i1529393", \  
"0.0255662, 0.0620065, 0.0989848, 0.1564231, 0.2948963, 0.5758429,  
i1433206", \  
"0.0396440, 0.0810312, 0.1165198, 0.1685752, 0.3027902, 0.5799232,  
i1402389", \  
"0.0660692, 0.1141829, 0.1540018, 0.2074403, 0.3319149, 0.5965588,  
i1514810", \  
"0.1137404, 0.1745757, 0.2194223, 0.2800810, 0.4078662, 0.6565910,  
i1786882", \  
"0.2058916, 0.2818740, 0.3383741, 0.4076877, 0.5491328, 0.8068217,  
i3029473", \  
"0.3963744, 0.4750280, 0.5496183, 0.6374584, 0.8049000, i0920195,  
i6171838");  
}
```

CMGen Setup File

A default CMGen setup file, `ctmgen.setup`, is provided with every CMGen release.

This file contains the following commands:

```
set log log_file  
set outdir output_directory  
set spice_model spice_library_model  
set cell_netlist cell_netlist_file  
set liberty_file liberty_file_name
```

See the [Invoking CMGen and the CMGen Setup Commands](#) section for a complete description of the commands you can use in the CMGen setup file.

3

Invoking CMGen and the CMGen Setup Commands

This chapter describes how to launch CMGen and the commands that comprise the CMGen setup file. The setup commands specify the required input files, and optional parameters, such as defect and vector type definitions, constraints, short, open defect, and voltage values. This chapter also describes a set of commands that you use to run CMGen on your SGE or LSF compute farm.

The following sections describe how to use invoke CMGen and the CMGen setup commands:

- [Invoking CMGen](#)
- [Example CMGen Setup File](#)
- [Required Setup Commands](#)
- [Optional Commands](#)
- [Compute Farm Commands](#)
- [Optional Farm-Specific Commands](#)
- [Setting Up Farm-Based Run Scripts](#)

Invoking CMGen

You launch CMGen using the `cmgen`. The following example runs CMGen based on the `ctmgen.setup` file:

```
% cmgen -setup ctmgen.setup
```

To list the available options, specify either the `-help` or `-man` options.

```
% cmgen -help
```

```
('Usage: ', 'cmgen')
-setupFile|-setup <SetupFileName>
-help #for help message
-man #for more detailed help message
```

Example CMGen Setup File

An example CMGen setup file is as follows:

```
set log_file cmgen.log
set output_dir ctm_out
set spice_model spice_model.inc
set cell_netlist AND2X1.spf
set liberty_file tt.25c.lib

## Optional Settings set keep_waveform 1
set numcores 8
```

Required Setup Commands

This section describes the required setup commands. These commands include:

- [set liberty_file](#)
- [set cell_netlist](#)
- [set spice_model](#)
- [set log_file](#)
- [set partition_multibit_netlist](#)
- [set output_dir](#)

set liberty_file

Description

Specifies the liberty file (.lib file), which contains the clock period, voltage, temperature, and pin information for each cell.

Syntax

```
set liberty_file libFileName
```

Examples

```
set liberty_file my_lib_file.lib
```

set cell_netlist

Description

Specifies an individual SPICE netlist. This command must be used independently of the `cell_list_file` command.

Syntax

```
set cell_netlist netlist_filename
```

Examples

```
set cell_netlist andgate.spx
```

set spice_model

Description

Specifies the location of the SPICE model file.

Syntax

```
set spice_model model_file
```

Examples

```
set spice_model spice.inc
```

set log_file

Description

Specifies the log file name and location.

Syntax

```
set log_file logFileName
```

Examples

```
set log_file cmtgen.xx.flat_nominal.default.spi.log
```

set partition_multibit_netlist

Description

Enables CMGen to identify subcells of a multibit cell.

By default, the option is set to true, which causes CMGen to partition the multibit cells into subcells.

Syntax

```
set partition_multibit_netlist true|false
```

Examples

```
set partition_multibit_netlist true
```

set output_dir

Description

Specifies the directory location for the output results.

Syntax

```
set output_dir outputDirName
```

Examples

```
set output_dir cmtgen.xx.flat_nominal.default.spi.log
```

set output_dir

Description

Specifies the directory location for the output results.

Syntax

```
set output_dir outputDirName
```

Examples

```
set output_dir cmtgen.xx.flat_nominal.default.spi.log
```

Optional Commands

Optional setup commands include the following:

- [set cell_ground](#)
- [set cell_list_file](#)
- [set cell_power](#)
- [set constrain_scan_enable](#)

- `set constrain_set_reset`
- `set defect`
- `set delay_variation`
- `set device_name_prefix`
- `set device_name`
- `set extra_hspice_options`
- `set extra_finesim_options`
- `set hspice_runelevel`
- `set hspice_wrapper_command`
- `set ignore_device_name`
- `set input_dir`
- `set keep_testbench`
- `set numcores`
- `set open`
- `set open_range`
- `set pin_constraint`
- `set pin_ignore`
- `set keep_waveform`
- `set short`
- `set short_range`
- `set skip_single_input_seq_cells`
- `set spice_resim_count`
- `set spice_simulator`
- `set threshold_voltage`
- `set temporary_storage`
- `set timing_margin`
- `set timing_resolution`

- `set tolerant_missing_tr_file`
- `set transistor_open_defect_device`
- `set transistor_open_defect_location`
- `set transistor_open_defect_resistor_value`
- `set transistor_short_defect_device`
- `set transistor_short_defect_location`
- `set transistor_short_defect_resistor_value`
- `set udd_dir_name`
- `set unspecified_device_handling`
- `set vector`
- `set verilog_netlist`
- `set voltage_variation`
- `set zip_waveform`
- `set include_transistor_instance`
- `set exclude_transistor_instance`
- `set create_ctm_for_0injection`
- `set create_ctm_for_0det`
- `set create_defect_csv file`
- `set create_exhaustive_ctm`
- `set outdir`
- `set merge_ctm_tables`
- `set skip_buffer_inverter_cells`
- `set keep_single_ctm_file`
- `set netlist_format DSPF`
- `set reuse_subcell_ctm`
- `set report_defect_detection_info`
- `set separate_static_dynamic_simulation`

- [set max_defect_by_user](#)
 - [set max_defects_per_simulation](#)
 - [set open_defect_class](#)
 - [set outdir_based_on_cellname](#)
 - [set spice_netlist_checker](#)
-

set cell_ground

Description

Sets the ground pin name(s) if not defined in the liberty file. Multiple pin names may be specified as a list.

Syntax

```
set cell_ground name
```

Examples

```
set cell_ground VSS GND
```

set cell_list_file

Description

Specifies a list of cells to process from the directory specified in the `set input_dir` command.

Syntax

```
set cell_list_file cell_list
```

Examples

```
set cell_list_file ./log120hdl090f.list
```

set cell_power

Description

Set the VDD pin name if it is not defined in the liberty file.

Syntax

```
set cell_power name
```

set constrain_scan_enable

Description

Constrains the scan enable pin for sequential elements to the off state. The default value is true.

Syntax

```
set constrain_scan_enable true|false
```

set constrain_set_reset

Description

Constrains the set/reset pin(s) for sequential elements to the off state. The default value is true.

Syntax

```
set constrain_set_reset true|false
```

set defect

Description

Defines the defect types for simulation. The default defines the short, open, and transistor faults. Other options include transistor short, transistor open, transistor short and open.

Syntax

```
set defect transistor|short|open|shortopen
```

set delay_variation

Description

Specifies the amount of time in picoseconds (?D) to ignore when performing defect classification. The number will always be rounded up to the next strobe value. This number should be defined as a multiple of the “timing_resolution” value. The default is 0.

Syntax

```
set delay_variation time_in_ps
```

set device_name_prefix

Description

Specifies the NMOS and PMOS transistor model prefixes. This command is only necessary when targeting specific transistor devices.

Syntax

```
set device_name_prefix model prefix
```

set device_name

Description

Defines the Spice-level transistor types for custom injections.

Syntax

```
set device_name [nmos | pmos] name
```

Examples

```
set device_name nmos hp3bn45
```

set extra_hspice_options

Description

Enables you to define additional HSPICE switches.

Syntax

```
set extra_hspice_options options
```

Examples

```
set extra_hspice_options -hpp
```

set extra_finesim_options

Description

Enables you to define additional FineSim command line options.

Syntax

```
set extra_finesim_options command_line_options
```

Examples

```
set extra_hspice_options -hpp
```

set hspice_runelevel

Description

Decreases the simulation time by reducing the accuracy of good AND defect simulation.

Syntax

```
set hspice_runelevel number
```

Examples

```
set hspice_runelevel 2
```

set hspice_wrapper_command

Description

Replaces the HSPICE executable for a custom path. No checking is performed.

Syntax

```
set hspice_wrapper_command path
```

Examples

```
set hspice_wrapper_command simulationa/run3
```

set ignore_device_name

Description

Specifies a device model prefix to be ignored for defect injection.

Syntax

```
set ignore_device_name model
```

set input_dir

Description

Specifies the directory location for the input SPICE netlist files.

Syntax

```
set input_dir inputDirName
```

Examples

```
set input_dir /remote/release/latest/spice/netlists
```

set keep_testbench

Description

Keeps only SPICE testbench files (1 = true, 0 = false). The default value is false.

Syntax

```
set keep_testbench [0 | 1]
```

Examples

```
set keep_testbench 1
```

set numcores

Description

Sets the number of cores to use for SPICE and waveform processing. The default value is 2.

Syntax

```
set numcores number
```

Examples

```
set numcores 8
```

set open

Description

Sets the open defect in ohms. The default is 1Gohm. Values may also be entered as a list.

Syntax

```
set open float
```

Examples

```
set open 1e9 100
```

set open_range

Description

Sets the open defect based on a range of values and a step size. The default value is 1000000000.

Syntax

```
set open_range start stop step_size
```

Examples

```
set open_range 1e9 1e3 10X
```

set pin_constraint

Description

Sets the constant values for the scan enable, set, and reset pins.

Syntax

```
set pin_constraint pin_name constant_value
```

Examples

```
set pin_constraint TICK 1
```

set pin_ignore

Description

Ignores the specified output pins.

Syntax

```
set pin_ignore pin_name
```

Examples

```
set pin_ignore p14
```

set keep_waveform

Description

Specifies to keep (1) or delete (0) HSPICE waveform files. The default is false.

Syntax

```
set keep_waveform [0|1]
```

set short

Description

Sets the short defect in ohms. The default is 1ohm. Values may also be entered as a list.

Syntax

```
set short float
```

Examples

```
set short 1 100
```

set short_range

Description

Sets the short defect based on a range of values and a step size.

Syntax

```
set short_range start stop step_size
```

Examples

```
set short_range 1 1e3 10X
```

set skip_single_input_seq_cells

Description

Allows CMGen to automatically skip processing sequential cells with single inputs.

Syntax

```
set skip_single_input_seq_cells true|false start stop step_size
```

Examples

```
set skip_single_input_seq_cells true
```

set spice_resim_count

Description

Specifies the number of times to resimulate when there are runtime errors. The default value is 1.

Syntax

```
set spice_resim_count number
```

Examples

```
set spice_resim_count 3
```

set spice_simulator

Description

Specifies between the FSIM or HSPICE simulators. The default value is HSPICE.

Syntax

```
set spice_simulator [finesim | hspice]
```

Examples

```
set spice_simulator finesim
```

set threshold_voltage

Description

Sets the threshold voltage margin as a percentage of VDD; 0.2 means 20%. Both VOL and VOH are defined equidistant from the ground and VDD, respectively. This command overrides the voltage threshold values specified in the liberty file.

Syntax

```
set threshold_voltage float
```

Examples

```
set threshold_voltage .2
```

set temporary_storage

Description

Uses temporary storage that enables CMGen to run on a machine's local disk. This command limits access to the machine's local disk, which reduces network traffic and disk access issues.

Syntax

```
set temporary_storage path
```

Examples

```
set temporary_storage /cmgen/temp
```

set timing_margin

Description

Specifies the additional clock period as a percentage of the clock period specified in the liberty file. For example, 0.1 means the clock period is 10% over the value specified in the liberty file. This is important for the detection of dynamic (transition) defects. The default is (0.8*Liberty_Period).

Syntax

```
set timing_margin float
```

Examples

```
set timing_margin .1
```

set timing_resolution

Description

Specifies the time (Dt) between waveform samples (strokes). This value affects the number of stored values in the .tr files. The default value is 5ps.

Syntax

```
set timing_resolution time_in_ps
```

Examples

```
set timing_resolution 6
```

set tolerant_missing_tr_file

Description

Allows CMGen to continue when there are missing .tr files

Syntax

```
set tolerant_missing_tr_file true|false
```

Examples

```
set tolerant_missing_tr_file true
```

set transistor_open_defect_device

Description

Targets open defects to device type. The default value is pmos_nmos.

Syntax

```
set transistor_open_defect_device [nmos | pmos | [gate_source |gate_drain  
| source_drain]
```

set transistor_open_defect_location

Description

Targets open defects to terminals. The default value is pmos_nmos.

Syntax

```
set transistor_open_defect_location [nmos | pmos | [gate | source |  
drain]]
```

set transistor_open_defect_resistor_value

Description

Sets the open resistance values for transistors independently of other parasitics.

Syntax

```
set transistor_open_defect_resistor_value val | start stop step
```

set transistor_short_defect_device

Description

Targets short defects to device type.

Syntax

```
set transistor_short_defect_device nmos | pmos
```

set transistor_short_defect_location

Description

Targets short defects to terminals.

Syntax

```
set transistor_short_defect_location nmos | pmos
```

set transistor_short_defect_resistor_value

Description

Sets the short resistance values for transistors independently of other parasitics.

Syntax

```
set transistor_short_defect_resistor_value val | start stop step
```

set udd_dir_name

Description

Specifies the user-defined defects directory. Each cell should have a corresponding defect configuration file in this directory.

Syntax

```
set udd_dir_name userDefinedDefectsDIR
```

Examples

```
set udd_dir_name /cmgen/udd
```

set unspecified_device_handling

Description

Specified message handling option when in the spice netlist, there was a device that was not defined to what type of device(for example, nmos or pmos). Default value is “ignore”, which means CMGen will ignore the unspecified device without warning or error messages. “warn”, CMGen prints a warning message then continues. “error”, CMGen prints an error message and returns an error.

Syntax

```
set unspecified_device_handling ignore|error|warn
```

Examples

```
set unspecified_dveice_handling ignore
```

set vector

Description

Specify the types of input vector created for dynamic defect table. Multiple input transition vectors allows two or more input pins to toggle at same time.

Syntax

```
set vector compact_single_input_transition|multiple_input_transition
```

Examples

```
set vector multiple_input_transition
```

set verilog_netlist

Description

Specifies the path and name of the Verilog cell representation netlist file needed for the CMGen validation flow.

Syntax

```
set verilog_netlist_path|filename
```

Examples

```
set verilog_netlist ./CMGen/ulvt_tmax.v
```

set voltage_variation

Description

Specifies the maximum allowable analog voltage variation (?v) from the good and defect-injected simulation. The default is 10% of VDD.

Syntax

```
set voltage_variation float
```

Examples

```
set voltage_variation .12
```

set zip_waveform

Description

Outputs the waveforms as zip files (1 = compressed, 0 = no compression). The default value is true.

Syntax

```
set zip_waveform [0 | 1]
```

Examples

```
set zip_waveform 1
```

set include_transistor_instance

Description

Sets the transistor instance names to inject defect.

Syntax

```
set include_transistor_instance transistor_name
```

Examples

```
set include_transistor_instance Xld_M134
```

set exclude_transistor_instance

Description

Sets the transistor instance names not to inject defect.

Syntax

```
set exclude_transistor_instance transistor_name
```

Examples

```
set exclude_transistor_instance Xld_M134
```

set create_ctm_for_0injection

Description

Creates a dummy CM file for a cell without any defect injection. The default value is true.

Syntax

```
set create_ctm_for_0injection true|false
```

Examples

```
set create_ctm_for_0injection true
```

set create_ctm_for_0det

Description

Creates a dummy CM file for a cell without any detected defect. The default value is true.

Syntax

```
set create_ctm_for_0det true|false
```

Examples

```
set create_ctm_for_0det true
```

set create_defect_csv file

Description

Create defect CSV file containing the CMGen Validation details for each cell.

Syntax

```
et create_defect_csv_file true|false
```

Examples

```
set create_defect_csv_file true
```

set create_exhaustive_ctm

Description

Create CM that contains every possible input vectors and arbitrary defect IDs.

Syntax

```
set create_exhaustive_ctm true|false
```

Examples

```
set create_exhaustive_ctm true
```

set outdir

Description

Specifies the directory location for the output result

Syntax

```
set outdir path
```

Examples

```
set outdir ./output
```

set merge_ctm_tables

Description

Merges `.CM` and `.resim.CM` into a single `.merge.CM` file at the end of the CMGen process. The default value is false.

Syntax

```
set merge_ctm_tables true|false
```

Examples

```
set merge_ctm_tables false
```

set skip_buffer_inverter_cells

Description

Allow CMGen to automatically skip processing buffer and inverter types of cells

Syntax

```
set skip_buffer_inverter_cells true|false
```

Examples

```
set skip_buffer_inverter_cells true
```

set keep_single_ctm_file

Description

Keeps a single `.merge.CM` file while deleting the `.CM` and `.resim.CM` files. The default value is `false`.

Syntax

```
set keep_single_ctm_file true|false
```

Examples

```
set keep_single_ctm_file false
```

set netlist_format DSPF

Description

Specifies that the physical information is in DSPF format extracted by Calibre.

Note:

Without this command, CMGen assumes that the physical information is in the StarRC (Synopsys) or HSPICE netlist format.

Syntax

```
set netlist_format DSPF
```

Examples

```
#rA/28 A A:23 0.0636316 $w=0.38 $l=0.31 $layer=m1Conn $X=0.28 $Y=1.96  
$X2=0.59  
#+$Y2=1.96
```

set reuse_subcell_ctm

Description

Reuses existing subcell level CTMs for multi-bit cells and saves runtime for large multi-bit cells.

Syntax

```
set reuse_subcell_ctm true|false
```

Examples

```
set reuse_subcell_ctm true
```

set report_defect_detection_info

Description

Writes detail defect detection information as comments in the CM file.

Syntax

```
set report_defect_detection_info true|false
```

Examples

```
set report_defect_detection_info true
```

set separate_static_dynamic_simulation

Description

By default, CMGen simulates static vectors and dynamic vectors in two separate simulation passes. With this command, user can turn off the static vector simulation.

Syntax

```
set separate_static_dynamic_simulation true|false
```

Examples

```
set separate_static_dynamic_simulation false
```

set max_defect_by_user

Description

Specify the number of defects to be simulated by CMGen. The default is to simulation all defects

Syntax

```
set max_defect_by_user N
```

Examples

```
set max_defect_by_user 100
```

set max_defects_per_simulation

Description

Specify the number of defects to undergo simulation per Hspice simulation.

Syntax

```
set max_defects_per_simulation N|20000
```

Examples

```
set max_defects_per_simulation 500
```

set open_defect_class

Description

set open defect to be classified as static, dynamic or both static and dynamic defects. By default, open defect is classified as dynamic defect only.

Syntax

```
set open_defect_class static|dynamic|both
```

Examples

```
set open_defect_class both
```

set outdir_based_on_cellname

Description

This command sets the output directory based on the actual cell name. The actual cell name uses the subckt name of the spice netlist. For example, spice netlist filename is "AND.spi" and subckt name is "AND" (.SUBCKT AND) in spice netlist, then the CTM output directory becomes [outputDirName]/AND.

By default, the directory set as [outputDirName]/AND_spi.

The default is false.

Syntax

```
set outdir_based_on_cellname true|false
```

Examples

```
set outdir_based_on_cellname true
```

set spice_netlist_checker

Description

This command is used to quickly check the Spice netlist files with a good simulation.

If there is a directory that contains the Spice netlist files, CMGen can use hspice to run good simulations for all cells. With this command, CMGen specifies the directory in which to perform a spice netlist check on the Spice netlist in that directory.

The default is false.

Syntax

```
set spice_netlist_checker
```

Examples

```
set spice_netlist_checker true | false  
set spice_netlist_checker | true
```

Compute Farm Commands

If your compute farm uses SGE or LSF grid or cloud processing, you should work directly with your IT organization to set the correct parameters for your particular implementation. The following compute farm commands are available:

set farm

Description

Enables processing on the compute farm (the default is false).

Syntax

```
set farm true|false
```

Examples

```
set farm true
```

set farm_type

Description

Sets the type of processing on the compute farm (the default is SGE).

Syntax

```
set farm_type SGE|LSF
```

Examples

```
set farm_type LSF
```

Optional Farm-Specific Commands

The following commands are dependent on the setup of your farm network. These commands may not work on all farm configurations.

set farm_option

Description

Limits the number of CPUs per job on a particular compute farm. The `-now` option notifies the queuing mechanism to wait if not the specified number of CPUs are not immediately available.

Syntax

```
set farm_option -pe -now yes|no
```

Examples

```
set farm_option -pe mt 8 -now no
```

set farm_project

Description

Specifies the name of the farm queue. Note that the `bnormal`, `heavy`, and `large` parameters may have different meanings depending on your compute farm.

Syntax

```
set farm_option bnormal|heavy|large
```

Examples

```
set farm_project bnormal
```

set farm_resource

Description

Specifies how much memory to set aside for each job, the name of the host machine, and the operating system of the compute farm.

Syntax

```
set farm_resource mem_free=space, hostname=path arch=arch_type
```

Examples

```
set farm_resource mem_free=32  
set farm_resource mem_free=32G,hostname=\my_machine77
```

The following example shows how much memory to set aside, the machine architecture, and the OS distribution. Note that this setup may severely limit the amount of machines on a given compute farm.

```
set farm_resource mem_free=32G,arch=glinux,os_distribution
```

set active_job_limit

Description

Limits the number of cells submitted to the farm.

Syntax

```
set active_job_limit number
```

Examples

```
set active_job_limit 50
```

Setting Up Farm-Based Run Scripts

The `set output_dir` command can also specify the location of a directory, called `farm.setup`, that contains all unique run scripts submitted to the farm. The following example shows a recommended initial setup:

```
set farm true  
set farm_type LSF  
set farm_option -now no  
set farm_project bnormal  
set farm_resource mem_free=8G,arch=glinux,os_distribution=redhat
```

4

Defect Site Selection

Defect selection is performed using the parasitics and devices present in the SPICE netlist. CMGen creates a simulation environment to target these defects using Synopsys SPICE simulators. The simulation results are then post-processed according to defined selection criteria to determine candidates for the cell test model.

The following sections explain the methodology used by CMGen to select defect locations:

- [R/C Defect Selection](#)
- [Transistor Defects](#)
- [User-Defined Defects](#)

R/C Defect Selection

By default, automated defect selection identifies all parasitic elements in a SPICE netlist. The CMGen RC model approximates physical geometries to their effective parasitic element. For example, a large capacitance might point to a pair of net geometries which are a potential bridging candidate.

There are two basic defect types:

- Short defect

This defect type is representative of internal cell nodal bridging. It has a very small resistance (one ohm of resistance, by default), which allows the current to easily flow across two nets or devices.

- Open defect

This defect type has a large resistance (one giga-ohm, by default). This defect impedes the flow of the current in either a broken net, a high resistive net, or an inline resistive on the node.

A subset of open defect is open_via defects. The following commands target only viaopen defects.

```
set defect open_via only
```

Enabling open_via only defect will automatically enable multiple_input_transition vectors

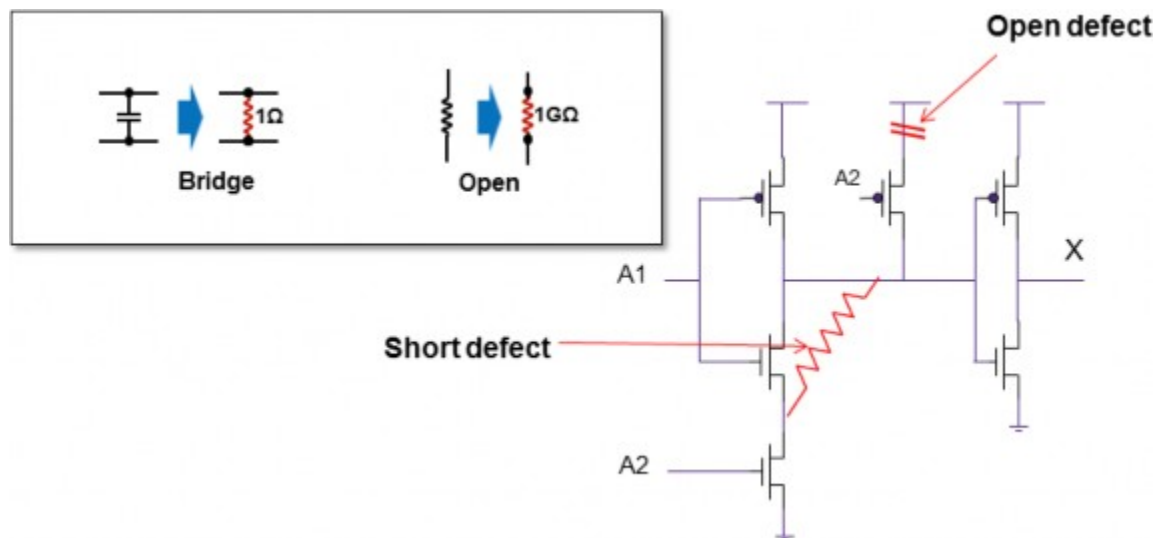
CMGen targeting via-open defects will only create dynamic vectors for detections. To also generate static detection vectors, use command:

```
set open_defect_class both
```

For defect selection, CMGen “shorts” each capacitor and “opens” each resistor.

Defect types include resistive open defects, capacitor shorts, transistor open or shorts, and userdefined defects. Figure 3 shows the bridge and open defects derived from parasitics.

Figure 3 Defects Derived from Parasitics



You can use the following commands to modify the values for open and short defects for both single and range values:

- `set short value`
- `set open value`
- `set short_range start stop step`
- `set open_range start stop step`

The set short and set open commands maybe be used with single or multiple values. Each value is simulated independently. The range allows for the resistance to be swept across a range and step size. For a complete description of each command, see [Required Setup Commands](#)

Note the following:

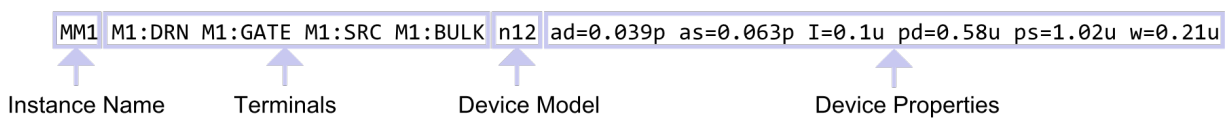
- During netlist generation, you can select to assign capacitance values below a specified threshold to a '0' node. While maintaining the parasitic presence in the netlist, this allows for simulator optimization by removing the effect from consideration. CMGen also honors this condition and avoids faulting devices connected to '0'.
- Changing the resistance values significantly impacts the behavior of the defect site. Ranges are useful for exhaustively characterizing the possible behaviors of a defect location, but using this specification increases the characterization time.

Transistor Defects

A standard representation of a transistor in the SPICE netlist is a 4-terminal device (drain, gate, source, and bulk). This includes both planar transistors and newer technologies such as FinFET. The SPICE library abstracts the underlying structure of the FET.

Figure 4 Example SPICE Netlist Transistor Syntax

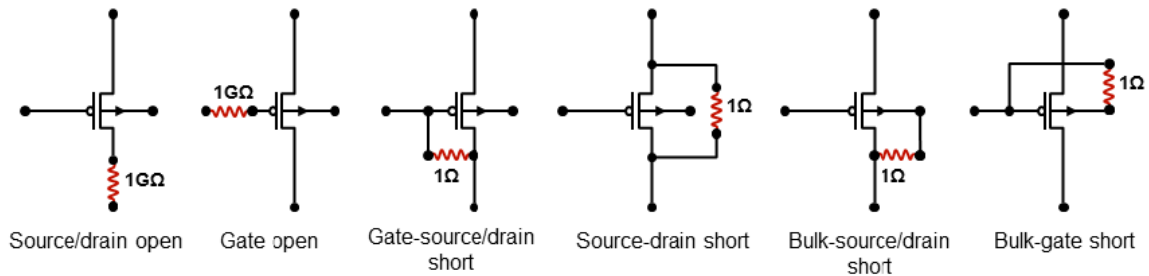
* Instance Section



Parasitics that represent the physical geometries of the terminals and their associated net or device connections may already be present in the SPICE netlist. However, due to various forms of RC network simplification, CMGen cannot guarantee the presence of any parasitic. At the risk of some known redundancy, CMGen introduces shorts and opens specifically at the terminals of each transistor to guarantee full defect coverage of the device. Default resistance values for open and short defects remain the global default defined for standard parasitics.

For transistor-level defects, each terminal of a resistor is shorted and open, as shown in Figure 4.

Figure 5 Transistor-Level Defects



By default, transistors are targeted together with all parasitics in the SPICE netlist. There are applications where targeting only the transistors is preferred. The 'set defect' command may be used to add only transistor defects.

The following command targets only transistor defects:

```
set defect transistor
```

Open and short resistance values may be set for transistors independently of other parasitics using the following commands:

```
set transistor_open_defect_resistor_value val | start stop step
```

```
set transistor_short_defect_resistor_value val | start stop step
```

CMGen does not differentiate between model types for injections. By defining the NMOS and PMOS device models, additional user controls are available.

Defines the NMOS and PMOS devices:

```
set device_name_prefix <pmos | nmos> device_model
```

Ignores devices in the netlist:

```
set ignore_device_name device_model
```

Targets open or short defects to device type:

```
set transistor_open_defect_device <nmos | pmos>
```

```
set transistor_short_defect_device <nmos | pmos>
```

Targets open or short defects to terminals:

```
set transistor_open_defect_location <source | gate | drain>
```

```
set transistor_short_defect_location <gate_source | gate_drain | source_drain>
```

Example CMGen.setup file to target NMOS source defects:

```
set log cmgen.log
set numCores 8
set libmodel process.spice
set liberty_file liberty.lib
set cell_netlist flop.spx
set device_name_prefix pmos pch_mac
set device_name_prefix nmos nch_mac
set transistor_open_defect_device nmos
set transistor_open_defect_location source
set transistor_open_defect_resistor_value 1 1e9 10x
```

User-Defined Defects

In addition to automated site selection, CMGen provides an interface to introduce user-defined defects into the model. User-defined defects may be run either independently or be appended to the default injection flow.

User-defined defects require design level knowledge of the cell. Each cell requires a unique user-defined defect (UDD) file.

The following topics describe user-defined defects:

- [Enabling User-Defined Defects](#)
- [Logical UDD File Syntax](#)
- [Physical UDD File Syntax](#)

Enabling User-Defined Defects

To enable user-defined defects, the following commands are required in the CMGen.setup file:

- `set defect udd`
- `set udd_dir_name location_of_UDD_files`

The `set defect` command is used as a list to enable user defined analysis.

The following example enables user-defined defects and the default injection flow in the same CMGen session:

```
set defect udd default
```

The `set udd_dir_name` command points to the folder containing the UDD files. Each file must be named for the cell to be characterized. For example, a cell named 'AND2X1' must

have a corresponding UDD file named 'AND2X1.udd'. There may only be one UDD file per cell, but multiple defects can be defined in a single file.

The following example shows the contents of a UDD file (AND2X1.udd):

```
Defect: SHORT_EXAMPLE
add short VDD:F1042 A1:F1043 1
# add short <net1> <net2> <short_value>

Defect: OPEN_EXAMPLE
add open R_VDD_F1042_XMPA1 VDD:F1042_XMPA1 1000000000
# add open <device> <net> <open_value>

Defect: COMPLEX_EXAMPLE
add open R_VDD_F1042_XMPA1 VDD:F1042_XMPA1 1000000000
add short VDD:F1042 A1:F1043 1
```

Logical UDD File Syntax

Logical defect injection requires the identification of nets or devices in the SPICE netlist to define customized injection points for simulation. Each defect in the UDD file is defined with a minimum of 2 lines.

- Defect Name

Defect: *UDD_name*

Appears in the description of the defect in the CM file

- Defect Type

- *add short net1 net2 short_value*

Adds a short between two logically defined nets in the SPICE netlist.

- *add open device net open_value*

Opens the terminal of a logically defined SPICE netlist device

Complex defects can be introduced by providing multiple opens, shorts, or a combination of both.

Physical UDD File Syntax

Physical defect injection requires physical layout knowledge of the cell. In addition, physical annotations must be present in the SPICE netlist.

Figure 6 StarRC Physically Annotated Extraction

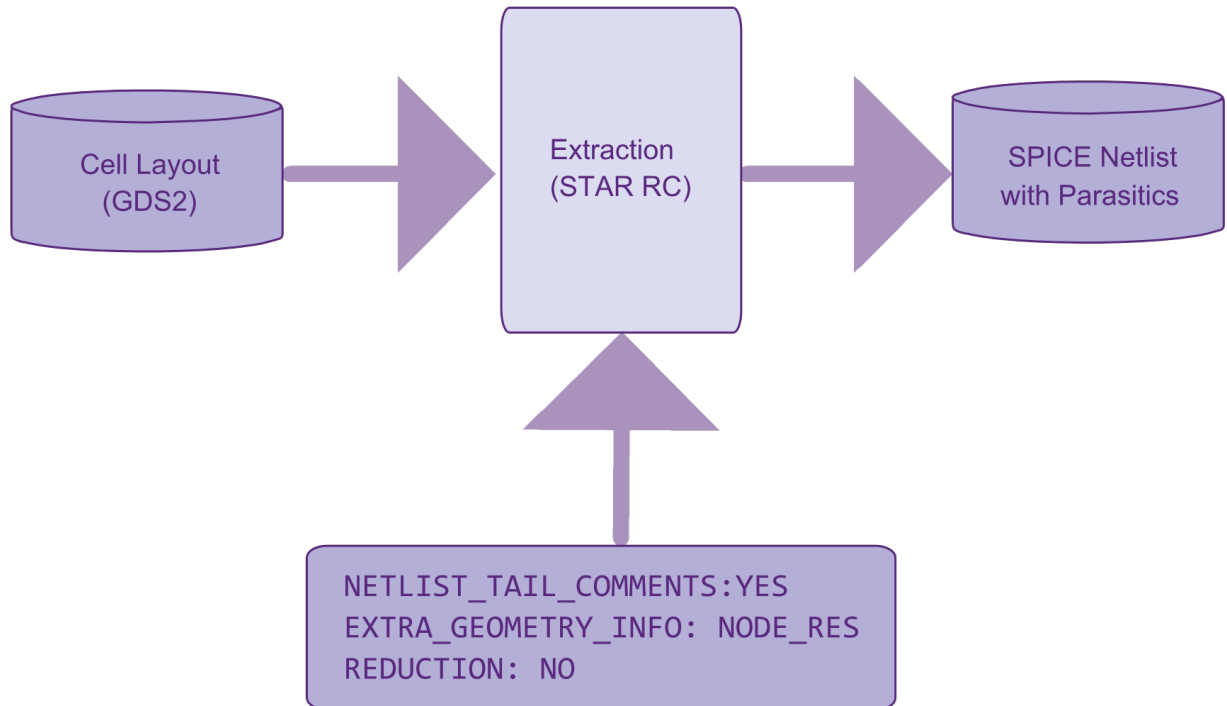


Figure 7 Example SPICE Netlist Syntax with Physical Annotations

```

SUBCKT AND2X1 VSS VDD IN2 IN1 Q
*LAYER_MAP
*0 SUBSTRATE
*1 nwnr
*2 Pwell
*3 nsd
*4 psd
*5 ntap
*6 ptap
*7 ngate_12
...
*|INET Q 0.000438127PF
*|P {Q B 0 2.34 0.855) // $l1x=2.12 $l1y=-0.68 $urx=2.36 $ury=0.865 $l1=50
*|1 (M6:SRC M6 SRC B 0 2.065 2) // $l1x=2.065 $l1y=1.44 $urx=2.37 $ury=2.56 $l1=4
*|1 (M3 DRN M3 DRN B 0 2.065 0.485)// $l1x=2.065 $l1y=0.285 $urx=2.37 $ury=0.685 $l1=3
Cg6 Q 0 1.1974e-17
Cg7 M6 SRC:0 4.3463e-17
Cg8 M3:DRN 0 6.47183e-19
R5 Q M6:SRC 3.14175
R6 0 M3:DRN 10.2314
...
  
```

Layer Map: Layers are database layer (mask layers)

Each net has physical bounding boxes and layers corresponding to the layer map (\$l1)

Parasitic elements may not have geometries assigned

The physically annotated SPICE netlist contains truncated physical data which can be used to approximate the region of interest. Each defect in the UDD file is defined with a minimum of two lines.

- Defect Name

Defect: *UDD_name*

Appears in the description of the defect in the CM file

- Defect Type

```
add short_defect_location <X1> <Y1> <TOTAL_CELL_AREA><LAYER>
```

Adds a short between the node contained at the specified coordinate to adjacent nodes. By default, the tolerance is set to 5% of the total cell area.

```
add short_defect_location <X1> <Y1> <X2> <Y2> <TOTAL_CELL_AREA><LAYER>
```

Adds a short between all nodes contained within the bounding box. By default, the tolerance is set to 5% of the total cell area.

```
add open_defect_location <X1> <Y1> <TOTAL_CELL_AREA><LAYER>
```

Opens the device terminal contained at the specified coordinate. By default, the search tolerance is set to 5% of the total cell area.

```
add short_defect_location <X1> <Y1> <X2> <Y2> <TOTAL_CELL_AREA><LAYER>
```

Opens all device terminals contained within the bounding box. By default, the search tolerance is set to 5% of the total cell area.

An example UDD file is as follows:

```
Defect: D1
add open XMMP6 vdd:F42 1e9
Defect: D2
add open XMMP6 vdd:F42 1e9
add short A2 A2:F98 1
```

The total cell area is used to approximate the search space for the location of physical defect sites. The search area tolerance can be set in the cmgen.setup file using the `set defect_search_range` command.

The following example extends the search range to 10% of the total cell area:

```
set defect_search_range 10
```

Coordinate data is relative to the origin defined in the SPICE netlist. The origin (0,0) is typically defined as the lower left corner.

5

Defect Detection

CMGen uses the behavior of the output waveform to determine the optimal ATPG model for targeting a defect:

- Static defects

The voltage does not stabilize to the threshold voltage for the duration of the simulation. There is no dependency on the previous vector state. Static defects are targeted in Stuck ATPG.

- Dynamic defects

The voltage can reach the good machine threshold with a delay. Also, all defects determined to have an input vector dependency are classified as dynamic. Dynamic defects are targeted in Transition ATPG.

All remaining defects are left unclassified. Transition ATPG does not target static defects. The following sections describes the methodology for defect detection:

- [Liberty Files for Simulation Input](#)
- [SPICE Output Files](#)
- [Testbench Files](#)
- [SPICE .lis File](#)
- [Waveform Analysis](#)
- [Multiple Output Support](#)
- [Sequential Cell Support](#)

Liberty Files for Simulation Input

CMGen uses the liberty (.lib) timing file to extract simulation setup parameters for the corresponding SPICE libraries. Liberty files are created through the SPICE characterization of the standard cell library. By replicating the original setup parameters from standard cell characterization, it can be assured that the CMGen simulation parameters are representative of the values expected from the standard cell designer.

All values necessary for HSPICE simulation are derived from the liberty file. For more information on the liberty file, see the [Liberty File](#) section.

SPICE Output Files

CMGen uses two output file formats to determine waveform behavior:

- Static measure file format (*.mt*) – verifies static defect behavior
- Dynamic transient file format (*.tr*) – verifies dynamic behavior.

Both formats are ASCII files which represent the analog voltage levels after the input stimulus is applied.

General output structure:

```
<output_log>.log : Full log file
<output_dir>/<cell>
<cell>.CM - CM model file
<cell>.output
<cell>.CMGen - Modified spice netlist
<cell>.CMGen.bench - Spice test bench
<cell>.CMGen.vec - Input vector file
<cell>.tr(n) - ASCII Transient Waveform files. .tr0 is good machine.
<cell>.mt(n) - ASCII Transient Waveform files. .mt0 is good machine.
<cell>.lis - SPICE list file. SPICE output log file
```

Caution:

CMGen does not interpolate the events between the captured simulation times. However, you can change these captured event times.

Static Measure Files

Static measure files are assigned names according to the number of defects in the simulation. The <cell>.mt0 file represents a defect-free circuit simulation. Each subsequent .mt file represents a defect. This numbering system matches the final CM file. These files are not preserved at the end of a CMGen run by default. To preserve the waveform files, use the following command:

```
set keep_waveform 1
```

Dynamic Transient Files

Dynamic transient files, like static measure files, are assigned names according to the number of defects in the simulation. The <cell>.tr0 file represents a defect-free circuit simulation. Each subsequent .tr file represents a defect. The filenames match the CM

defect ID. By default, these files are not preserved at the end of a CMGen run. To preserve the waveforms, use the following command:

```
set keep_waveform 1
```

Testbench Files

The SPICE testbench is composed of three files:

- `<cell>.CMGen` – The SPICE netlist which has been modified for parameterized parasitics
- `<cell>.CMGen.bench` – The SPICE file used to drive the simulation
- `<cell>.vec` – The file containing the input values during simulation

By default, these files are not preserved at the end of a CMGen run. Use the following command to preserve test bench files when saving the waveforms:

```
set keep_waveform 1
```

Alternatively, you can use the following command to preserve only the testbench files:

```
set keep_testbench 1
```

SPICE .lis File

The .lis file is a SPICE generated log file. If an unsuccessful SPICE simulation occurs, the .lis file is useful for debugging the simulation setup. The .lis file(s) can be reviewed for error messages and warnings.

Waveform Analysis

When evaluating the simulation data, CMGen attempts to extract three primary criteria:

- Detection status – a holistic view of determining if a defect is detected.
- Detection type – the model designation of the defect (Static or Transition).
- Uniqueness – the input combinations required to trigger the defective behavior. If the defect has a unique input signature, it is classified as unique in the CM model.

Caution:

Defects can exhibit analog behavior which may appear unique in a simulator, but digitally there is no way to uniquely classify the defect.

Static Defect Analysis

Each defect is first evaluated at each unique vector in the vector set. In the absence of any previous setup conditions, the output behavior is compared to the defect-free circuit to check for deviations. The results can be observed in the static measure output files.

A file with a *.mt0 extension represents a defect-free circuit and each successive file represents a defect using the same numbered naming convention. By default, a detection is registered when the analog voltage value difference exceeds 10% of the device rail voltage. In addition to the default behavior, there are two commands for controlling the static detection status:

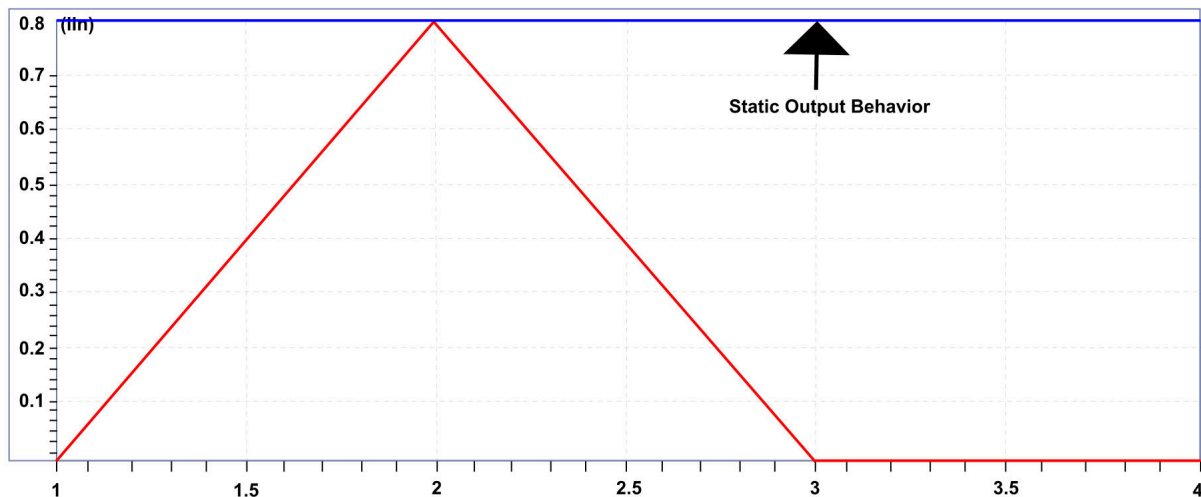
- `set voltage_variation num_percent_vdd`

Sets the default voltage variation to consider a detection. The default is 10%. This setting also effects dynamic detection consideration.

- `set voltage_variation_for_static_detection < true | false >`

When set true, static detection will use the voltage variation percentage to decide a detection. When set false, a detection is only registered when the output value exceeds a digital threshold (0, X, 1).

To be considered a static defect, a defect must be detectable with at least one vector. Defects that are initially classified as static can still have the detection status nullified by the behavior observed in the full dynamic vector set. All defects, regardless of static classification, are compared again in the full vector set to confirm the behavior.



Dynamic Defect Analysis

Dynamic defect analysis requires transition vectors to verify output behavior. The vector set is applied to the cell for the defect free circuit (*.tr0) and each subsequent defect simulation is compared for deviations in analog voltage (*.tr<n>).

CMGen setup variables impact the detection and classification of defects. There are four primary settings which control the dynamic simulation and decision making as it relates to classification:

- `set timing_margin float`

The additional time applied to the total simulation period to observe output behavior (the default is 30%)

- `set timing_resolution interval`

The interval between compare points (the default is 5ps).

- `set voltage_variation interval`

The voltage difference as a percentage of VDD between the defect and defect free simulation (the default is 10%).

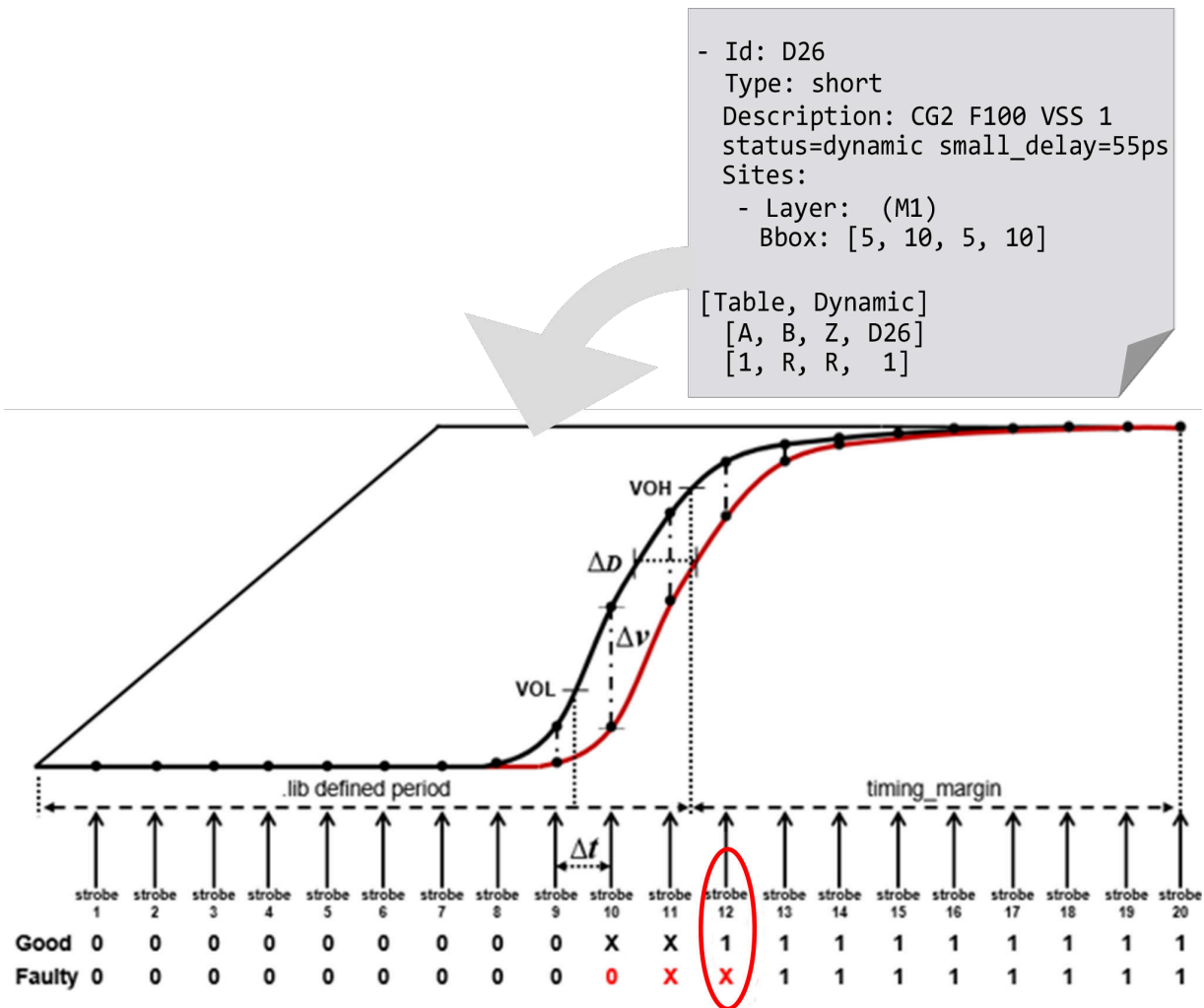
- `set delay_variation interval`

The minimum time interval for defect classification (the default is 0).

CMGen determines defects by sweeping an entire simulation period. By default, CMGen uses a 5ps interval between comparison points.

The `set timing_resolution` command improves defect resolution by controlling the time between comparison points. The following figure shows the results of waveform sweeping based on a 2ps interval. The black line is the good circuit and the red line is the faulty circuit. A defect is identified if the voltage difference on each strobe between these two lines is larger than the voltage variation specified by the `set voltage_variation` command (indicated in the figure as Dn.).

Figure 8 Output Sampling



Multiple Output Support

For multiple outputs, CM files are annotated with an integer to determine failing output combinations. This integer value is representative of a binary value description of the output pins, with the least significant bit representing the last output entry in the CM table. The following example table uses outputs S and CO. A value of 0 (00) represents no detection, a 1 (01) is detection by CO, a 2 (10) is detection by S, and a 3 (11) is detection by both S and CO.

```
[A,B,CI, S,CO, D1,D2,D3,D4,D14,D181]
0] -> Defect detected on S
0] -> Defect detected on both CO 0]
```

Sequential Cell Support

Sequential cell support is currently limited to flip flops and latches. When given a list of cells, CMGen will automatically exclude any unsupported cell(s) from characterization. The user will be notified of this operation at the end of the CMGen run. All required settings are extracted from the liberty timing files.

By default, the scan enable and set/reset signals are constrained to the off value. This behavior can be changed by using the following commands in the cmgen.setup file:

`set constrain_scan_enable true | false`

`set constrain_set_reset true | false`

The resulting CM file contains detection information which is unique to the sequential cell type. Clocks are defined as 'P' or '0'. The 'P' represents the active state of the clock and '0' is representative of the clock off state. Additionally, each output pin contains a previous state value denoted by a '-'.

The following example shows a sequential cell detection table as it appears in a CM:

Figure 9 Example - Sequential Cell Detection

```
- [A, B,CI, S,CO, D1,D5,D6,D7,D9,D10,D18]
- [0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 1]
- [0, 0, 1, 1, 0, 0, 0, 0, 3, 0, 0, 1]
- [0, 1, 0, 1, 0, 0, 0, 0, 0, 2, 0, 1]
- [0, 1, 1, 0, 1, 3, 0, 0, 3, 0, 0, 1]
- [1, 0, 0, 1, 0, 0, 0, 3, 0, 0, 0, 1]
- [1, 0, 1, 0, 1, 3, 2, 0, 0, 0, 0, 1]
- [1, 1, 0, 0, 1, 0, 0, 3, 0, 0, 0, 1]
- [1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 2, 1]

1(01) -> Defect detected on S
2(10) -> Defect detected on CO
3(11) -> Defect detected on both S and CO
```

6

Simulation Performance Turning

The overall runtime of a standard cell is generally a function of pin count, transistor count, parasitic count, and simulation accuracy. By default, CMGen is tuned for accuracy at the expense of increased runtime.

Depending on the final application of the cell-aware model, a reduction in overall model accuracy might be acceptable in exchange for an overall decrease in library runtime. The following sections describe possible strategies for decreasing library runtime.

- [Machine Resource Provisioning](#)
- [Defect Targeting](#)
- [SPICE Simulator Accuracy](#)
- [Pin Count](#)

Machine Resource Provisioning

CMGen requires a minimum of four cores for reasonable runtime. Best practice is to provide as many cores as machine availability and site licensing allows. Most libraries can be characterized within a 32GB RAM allotment per cell. Simulation waveforms can require peak temporary storage of greater than 500GB. A fast disk is recommended. Network storage can add additional latency so for best performance use a fast, local disk.

The following command enables CMGen to use a temporary storage location for simulation waveforms:

```
set temporary_storage location
```

Simulation files are written to the specified location for evaluation. The final results are subsequently transferred to the run directory when completed.

Defect Targeting

Runtime can be significantly decreased by reducing the number of defects targeted. By default CMGen targets the following defect classes:

- Shorts– All capacitors in the design
- Opens– All Resistors in the design
- Transistors – All transistors in the design

You can use the `set defect` command as a list to enable defect classes.

The following example targets only transistor defects:

```
set defect transistor
```

SPICE Simulator Accuracy

By default, CMGen uses the highest level of accuracy recommended by Synopsys HSPICE and FineSim.

```
set hspice_runlevel <1 - 5> Default is 5
set finesim_simulator <options>
set timing_resolution <int> Default 5ps
```

Pin Count

Input pin count is one of the largest contributors to cell runtime. By default, CMGen creates single input transition vectors. This means for each input pin, only one input pin is permitted to change value per two vector sequence. This is similar to Liberty timing, which is typically annotated with only single pin timing arcs. The CMGen algorithm exhaustively targets these vectors and introduces additional padding vectors when necessary to guarantee single input only transitions.

You can use the following basic formula for calculating vector runtime:

$$(2^{(\text{num inputs})} + \text{padding_vecs}) * \text{Liberty_period} + \text{Static_vecs}$$

For a given standard cell with N input pins, the estimated number of vectors to simulate (N_{vector}) is calculated as follows:

$$N_{\text{vector}} = 2^N + 2 * N * 2^N + \text{padding_vectors}$$

In this estimation, the 2^N parameter is the number of static vectors. The $2 * N * 2^N$ parameters indicate the number of dynamic vectors. Some padding vectors could be

added to the test vectors to make sure there is only single transition between two adjacent vectors.

For each vector, the vector period for SPICE simulation is calculated as:

$$\text{Vector_period} = (1 + \text{timing_margin}) * \text{liberty_period}$$

For a library cell with N_{defect} number of defects, the SPICE simulator simulates all the vectors once for each defect. In this case, you can estimate the total CPU time to simulate the cell using the following formula:

$$T_{\text{total}} = (1 + N_{\text{defect}}) * N_{\text{vector}} * T_{\text{vector}}$$

Where T_{vector} is the CPU time to simulate a single vector.

The CPU time to simulate each vector T_{vector} is related the number of SPICE elements in a cell. This means the “size” of the library cell impacts the total CPU time. Note that the number of defects (N_{defect}) is also related to the number of elements in a cell. This means the size of a library cell has significant impact on the overall runtime.

By removing input transitions, you can significantly decrease cell runtime:

- `set pin_constraint pin value`

Sets a value constraint on an input pin

- `set constrain_set_reset true`

Uses liberty parsing to automatically constrain set/reset pins to the off state for sequential elements

- `set constrain_scan_enable true`

Uses liberty parsing to automatically constrain the scan enable pins to the off state (functional path only)

You can also improve the overall runtime by disregarding output pin behavior:

- `set pin_ignore pin_name`

Ignores output pin values

7

CM File Description

A CM file has the following sections:

- [Header](#)
- [Defect Description](#)
- [Defect Detection Tables](#)

Header

The header contains annotations that describe the various simulation settings in the model, such as the CMGen version, the voltage, temperature, etc. These annotations are informational only and are not used for TetraMAX fault modeling.

Example CM header:

```
# Cell Test Model Generator (or CMGen) is a software program that
# directs a Synopsys HSPICE\OOAE product simulator or other
# similar simulator to create cell-aware test models. The CMGen
# and its output (including, without limitation, models, text
# files and log files) are licensed to you solely for use with a
# Synopsys Licensed Product.
#
# Date: 2017-07-26 15:34:06.620236
# CTGen version : 2017-07-25
# HSPICE executable: /global/apps/hspice_2017.03-SP1-1/hspice/ bin/hspice
# HSPICE run level: 5
# Vector type: compact_single_input_transition
# Lib cell revision: 1.00000000
# Clock period (Launch-capture window): 0.775ns
# Load capacitance: 5.2e-14f
# Temperature: 25.0000000C
# Voltage: 1.2v
# Threshold voltage: low=0.12v high=1.08v
# Static voltage variation: true
# Open defect size: ['1000000000']ohm
# Short defect size: ('1'] ohm
# Transistor_open defect size: ('1000000000'] ohm
# Transistor_short defect size: ('1'] ohm
# PVT: vlp2t25p0000000
# Liberty period: 0.428423641426ns
```

```
# Timing margin: 0.80895712
# Delta delay: 5ps
# Voltage variation: 0.12v
# Total_defect=661 static_det=411 dynamic_det=128 defect_coverage=81.5%
```

Defect Description

The defect description is a mandatory section that contains detailed information of the cell and defects:

- **Id** – Each defect has a unique ID. For example “D1” or “D2”.
- **Type** – The defect type (or defect model) can be a short, open, transistor-short, transistor-open, etc.
- **Description** – A short description of the defect location, size (resistor values), and detection status.
- **Instance** – The instance of the defect shows which SPICE element (resistor, capacitor or transistor) is associated with the defect.
- **Sites** – The defect sites describe the physical layout information, which includes the layer and bounding box. A single defect can contain multiple layers and bounding-boxes.
- **Attributes** – The attributes of a defect contain the equivalent defect ID, and the defect behavior (stuck, delay or small-delay), and the detection class (DT: detected, UD:undetected, and PD: potentially detected).

The following example shows a defect description:

```
- Id: D6
Type: short
Description: C14 M12:DRN M12:Gate 1 status=static_det
Instance: C14
Sites:
- Layer: nsd (3)
Bbox: [605, 545, 905, 775]
- Layer: ngate_12 (7)
Bbox: [905, 545, 1004, 775]
Attributes:
- Class: DT
- DetEquivalent: D3
- Behavior: stuck
```

Defect Detection Tables

Defect detection tables describe the defect detection conditions. In most cases, there are two detection tables: one for static defect detection and one for dynamic defect detection.

In each detection table, the first columns contain the input and output pin values. The remaining columns contain the detection status of each defect. The pin values can be “0”, “1”, “R” (0 to 1 transition), “F” (1 to 0 transition), “P” (clock pulse), “X” (unknown). The detection status could be “0” (not detected by the corresponding input values) or “1” (detected by the corresponding input values).

The following example shows a defect detection section with two detection tables:

```
Detections:
- [Table, Static]
-[ IN1,IN2,IN3,Q,D2,D3,D10,D18,D19,D26,D30,D31,D32,D61,D66,D96,D105,D106,D122,D127,D130,D131,D135,D196,D206,D210,D245,D258,D302,D446,D561,D579]
- [ 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0, 1, 1, 1, 0, 1, 0, 0, 0]
- [ 1, 0, 1, 0, 1, 0, 0]
- [ 0, 0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 1, 0, 1, 1, 0, 0, 0]
- [ 1, 0, 1, 1, 0, 0]
- [ 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0]
- [ 1, 0, 0, 1, 1, 0]
- [ 0, 1, 1, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0]
- [ 1, 0, 1, 0, 0, 0]
- [ 1, 0, 0, 1, 0, 0, 1]
- [ 1, 0, 0, 0, 0, 1]
- [ 1, 0, 0, 1, 0, 0, 0]
- [ 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 1, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0, 1, 1, 1, 0, 0]
- [ 1, 0, 0, 0, 0, 1]
- [ 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0]
- [ 1, 0, 1, 0, 0, 0]
- [Table, Dynamic]
-
[ IN1,IN2,IN3,Q,D11,D22,D23,D35,D67,D78,D87,D88,D89,D97,D101,D190,D208,D209,D218,D327,D328,D331,D332,D356,D357,D358,D364,D368,D377,D379,D384,D395,D396,D4]
- [ 0, 0, R, R, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0]
- [ 0, 1, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, R, 0, R, 1, 1, 0, 1, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0]
- [ 0, 1, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ R, 0, 0, R, 1, 1, 1, 1, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 0, F, F, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, R, 1, F, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0]
- [ 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ R, 0, 1, F, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, F, 0, F, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 1, R, F, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 1, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ R, 1, 0, F, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 0, 1, 0, 0, 0, 0, 0, 1, 1, 1, 0, 0, 0]
- [ 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, F, 1, R, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
- [ 0, 1, F, R, 0, 0, 0, 0, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 0,
  0, 0, 1, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
- [ R, 1, 1, R, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1, 1, 0,
  1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1]
- [ F, 0, 0, F, 0, 0, 0, 0, 1, 1, 0, 0, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1,
  0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0]
- [ 1, 0, R, F, 1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1,
  0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 0, 1, 0, 0]
- [ 1, R, 0, F, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 1, 1,
  1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 1]
- [ F, 0, 1, R, 0, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0,
  0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 0, 0, 0]
- [ 1, 0, F, R, 0, 0, 0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0,
  0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0]
- [ 1, R, 1, R, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 1, 1, 0,
  1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1]
- [ F, 1, 0, R, 1, 0, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 1, 0,
  0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0]
- [ 1, F, 0, R, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0,
  0, 0, 1, 0, 1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0]
- [ 1, 1, R, R, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0,
  1, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 1]
- [ F, 1, 1, F, 1, 1, 1, 1, 0, 1, 0, 0, 1, 1, 1, 0, 1, 0, 0, 1, 0, 0, 1, 0, 0, 1, 1, 1,
  0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 1, 0, 0]
- [ 1, F, 1, F, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 1, 1, 0, 1, 0, 0, 1,
  0, 1, 0, 0, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 0, 0]
- [ 1, 1, F, F, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1,
  1, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1]
```

8

CM Validation Flow

The cell test model (CM) validation flow provides a methodology for validating CMs produced by CMGen (for more information, see the “Running CMGen” document). The following sections describe the CM validation flow:

- [Introduction](#)
- [CM Validation Specific Commands](#)
- [CNOD Specific Commands](#)
- [Example of a cmval.setup Run Script](#)
- [Generating the Output](#)

Introduction

You can use CMGen to perform a single cell ATPG validation in which each excitation row of a CM is verified for coverage in ATPG. Passing status requires 100% coverage.

Models are validated for the following:

- TetraMAX parsing
- Fault targeting
- Excitation state correctness

For validation purposes, pins are treated as ports.

The syntax for running CMGen in a standalone validation flow is as follows: `cmgen -ctm_validation_only -setup setup_file`

CM Validation Specific Commands

CMGen uses a setup file to set parameters for the validation. There are several required commands that must go in the validation flow setup file:

- `liberty_file` — Specifies the liberty file (.lib file), which contains the clock period, voltage, temperature, and pin information for each cell. For example:

```
set liberty_file ./CMGen_1105/cworst.lib
```

- `log_file` — Specifies the log file name and location. For example:

```
set log_file ./output/val.log
```

- `output_dir` — Specifies the directory location for the output results files. For example:

```
set output_dir ./output
```

- `verilog_netlist` — Specifies the path and name of the Verilog cell representation netlist file. For example:

```
set verilog_netlist ./CMGen_1105/ulvt_tmax.v
```

- `csv_file_data` — Collects the runtime results in CSV format. For example:

```
set csv_file pm.ulvt.csv
```

- `enable_atpg_validation` — Enables ATPG. For example:

```
set enable_atpg_validation true
```

Optional commands include:

- `validation_clock_offstate` — Specifies the clock offset by pin name. For example:

```
set validation_clock_offstate 1 CP
```

```
set validation_clock_offstate 0 CPN
```

- `remove_validation_logs` — Specifies the removal of the CM validation log files. By default, these files are not removed. The following example removes the validation log files:

```
set remove_validation_logs true
```

- `validation_pin_constraint` — Specifies the pin constraint values by pin name. For example:

```
set validation_pin_constraint 1 SDN
```

- `validation_capture_cycle` — Specifies the capture cycle number. For example:

```
set validation_capture_cycle 4
```

An example setup file is as follows:

```
set log_file ./output/val.log
set output_dir ./output
set
  liberty_file ./
CMGen_1105/tcbln05/tcbln05_bwph.lib./CTMGen_1105/tcbln05/tcbln05_nldm.lib
set verilog_netlist ././CMGen_1105/tcbln05/tcbln05_tmax.v
set csv_file pm.ulvt.csv
#set ctm_file_dir ./CMGen_1105_StarRC_XY_validation_2018_1112/pm_ulvt
set ctm_file_dir ./output/RSSDFSDRPQHDCW_BLLL_D1_H210_L6_P51_UL_spi
set validation_pin_constraint 1 SDN
set validation_pin_constraint 1 RETN
set validation_pin_constraint 0 CD
set validation_clock_offstate 1 CP
set validation_clock_offstate 0 E
set remove_validation_logs true
```

CNOD Specific Commands

This section describes the Continuous Diffusion (CNOD) commands. These commands include:

- [set cnod_defect_location](#)
- [set enable_cnod_detection_table](#)
- [set cell_instance_file](#)

set cnod_defect_location

Description

Sets the power or ground to connect a floating pin of MPODE transistor.

Syntax

Where 1 indicates connect PMOS drain to VSS, source to VDD, NMOS drain to VDD, source to VSS. 2 indicates connect PMOS drain or source to VDD and VSS, NMOS drain or source to VDD and VSS. The default value is 2.

```
set cnod_defect_location 1|2
```

Examples

```
set cnod_defect_location 2
```

set enable_cnod_detection_table

Description

Enables CMGen to write CNOD detection table in the CM file.

Syntax

Where, true indicates that CMGen creates the CNOD defect detection table in the CM file. The default value is false.

```
set enable_cnod_detection_table true|false
```

Examples

```
set enable_cnod_detection_table false
```

set cell_instance_file

Description

Sets the file that contains the list of cells to characterize

Syntax

```
set cell_instance_file cell_instance_file_name
```

Examples

```
set cell_instance_file user_defined_instance.lst
```

Example of a cmval.setup Run Script

The following is an example of a cmval.setup run script:

```
set log_file <file_name>  
set output_dir <path_name>  
set liberty_file <libFileName>
```

```
set verilog_netlist <file_name>
set ctm_file_dir <path_name>
set csv_file <file_name>
set validation_clock_offstate 0/1 <pin_name>
set validation_pin_constraint 0/1 <pin_name>
set validation_capture_cycle <#>
set remove_validation_logs true
```

Generating the Output

To generate the validation output specify the `cmgen` command, the `-ctm_validation_only` option, and the name of the setup file, as shown in the following example:

```
cmgen -ctm_validation_only -setup c5_setup_file
```

CMGen produces the following output files:

- A CSV file containing validation details for each cell (see the example below).
- A file named `ctm_validation.csv.pass.list` containing a list of all passing (100% coverage) cells.
- A file named `ctm_validation.csv.fail.list` containing a list of all failing (<100% coverage) cells.
- A file named `ctm_validation.csv.noctm.list` containing a list of all cells without CTMs.

An example CSV file:

```
cell_name, #SAF, #SA_coverage, #TF, #TF_coverage, #CTMValidation, DateTime
CKBUFDIDO_M10_D8_H219_L6_P51_UL, 0, 0, 5, 100.00%, Pass, 2018_12_10
13:59:
CKBUF_M19_D8_H219_L6_P51_UL, 9, 9, 2, 100.00%, Pass, 2018_12_10 14:96:32
CKBUF_M19UFDI_D4_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 14:14:91
CKBUF_M4_D64_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 14:25:99
CKBUFSIDO_M19_D64_H219_L6_P51_UL, 9, 9, 2, 100.00%, Pass, 2018_12_10
15:91
CKLNQ_M19_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 15:17:15
CKLNQ_M19SKF_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 15:32:25
CKLNQ_M19SKR_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 15:47:17
C1MUX2_M19BALD_D16_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 16:99:45
C1MUX2_M4BALS_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 16:19:34
C1MUX3_M19BALD_D8_H219_L6_P51_UL, 9, 9, 4, 100.00%, Pass, 2019_12_19
16:19:34
C1MUX4_M4BA_D8_H219_L6_P51_UL, 9, 9, 10, 100.00%, Pass, 2019_12_19
16:19:34
CKN_M19_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 21:58:59
CKN_M19SKF_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 22:92:23
CKN_M19SKR_D8_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 22:96:93
CKN_M4_D16_H219_L6_P51_UL, 9, 9, 9, 9, NA, 2018_12_10 22:99:21
```